Resumen para exponer Garritas

En 45 segundos: “Garritas es una tienda y plataforma de cuidado de mascotas. Angular usa Gateway para la API; Keycloak autentica. Los ocho microservicios tienen bases independientes. Las consultas inmediatas usan HTTP; la compra se resuelve por Kafka con Saga, idempotencia y compensación”.

Integrante	Transaccional	Maestro y N:M
Tú	Venta / DetalleVenta	Institución: sedes ↔ prestaciones
Compañera	Cita / DetalleCita	Envío: transportistas ↔ zonas
Fio	Usuario: suscripción / períodos	Catálogo como maestro; N:M cliente ↔ plan en Usuario

Flujo que debes poder contar

```
Angular → Gateway → Venta valida sede y productos
Venta + Outbox → Kafka → Catálogo reserva stock
Pago aprobado → Kafka → Venta PAGADA
Pago fallido → Catálogo devuelve stock → Venta CANCELADA
```

Un 201 de creación no significa pago confirmado. Angular hace polling. Compensar devuelve inventario y conserva historia; no borra la operación ni hace un reembolso bancario real.

Los tres archivos N:M para abrir en VS Code

Tu pareja: dos @ManyToOne con precio/duración · línea 17

`services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/entity/SedePrestacion.java`

Compañera: dos @ManyToOne con tarifa/plazo · línea 17

`services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/entity/Cobertura.java`

Fio: cliente y plan; períodos como detalle · línea 18

`services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/entity/Suscripcion.java`

Una tabla de asociación con dos N:1 representa N:M cuando ambos extremos pueden repetirse. Bienestar tiene además @ManyToMany explícito en entity/Profesional.java. No existe una FK que una bases distintas.

Para enseñar un CRUD

Controller: ruta/verbo → Service: regla → Repository: persistencia → Entity/migración: restricciones → Angular: resultado. Venta y Pago no exponen edición/borrado libre; sus estados pertenecen al proceso.

El despacho de compras pertenece a Venta. Envío ahora administra logística. Usuario conserva perfil/mascotas e incorpora contratación/renovación/cancelación de planes.

Siete respuestas cortas para recordar

1. Servicio remoto caído

Los timeouts limitan cuánto espero. En Venta → Catálogo, el Circuit Breaker deja de insistir tras suficientes fallos y el fallback responde controladamente. Feign usa nombre Eureka, conexión 2 s y lectura 3 s; el breaker no reemplaza al timeout.

2. Seguridad solo en Gateway

No basta: podrían llamar al backend directamente. Cada servicio valida el JWT y aplica rol y propiedad. En una ruta protegida: sin token válido, 401; con token válido pero sin permiso, 403. Las consultas públicas son excepciones intencionales.

3. Eventos frente a REST

Gano desacople temporal y puedo procesar cuando el consumidor vuelva. Pierdo el resultado final inmediato: aparecen pendientes, reintentos y posibles duplicados. Por eso uso consistencia eventual e idempotencia; no reemplazo todas las consultas REST por eventos.

4. Fallo a mitad del proceso

La Saga confirma pasos locales y compensa lo necesario: si falla el pago, restaura inventario una vez antes de cancelar. Inbox y restricciones únicas evitan duplicar efectos; Outbox conserva el evento. No hay un ACID global ni entrega exactamente una vez.

5. Evidencia de observabilidad

Grafana ubica la instancia con caída o cambios en HTTP/PostgreSQL. Prometheus permite comprobar targets/errores. Con un mismo X-Trace-ID, Loki muestra los mensajes relacionados de varios servicios en una sola consulta. up=1 no demuestra que una compra haya terminado.

6. Frontend sin URL de instancias

Angular usa una entrada estable, Gateway. Eureka y LoadBalancer resuelven instancias y cambios de cantidad/puerto. Así no acoplo el navegador al despliegue interno. La URL de Keycloak sí se conoce para autenticar; no es una instancia de negocio.

7. Reemplazo de auth por Keycloak

Si los claims/roles son compatibles, se conservan Resource Server y reglas. Cambian issuer/JWKS y el cliente OIDC. Además, el dueño idCliente pasa de Long a String/sub y requiere migración de código/datos. Garritas ya usa Keycloak: no afirma tener un auth propio temporal.

Recuerda: password grant no es el flujo usado. La SPA es pública, usa Authorization Code + PKCE y mantiene tokens en memoria, no en localStorage. El carrito sí guarda datos no sensibles. El pago del laboratorio sigue siendo simulado.